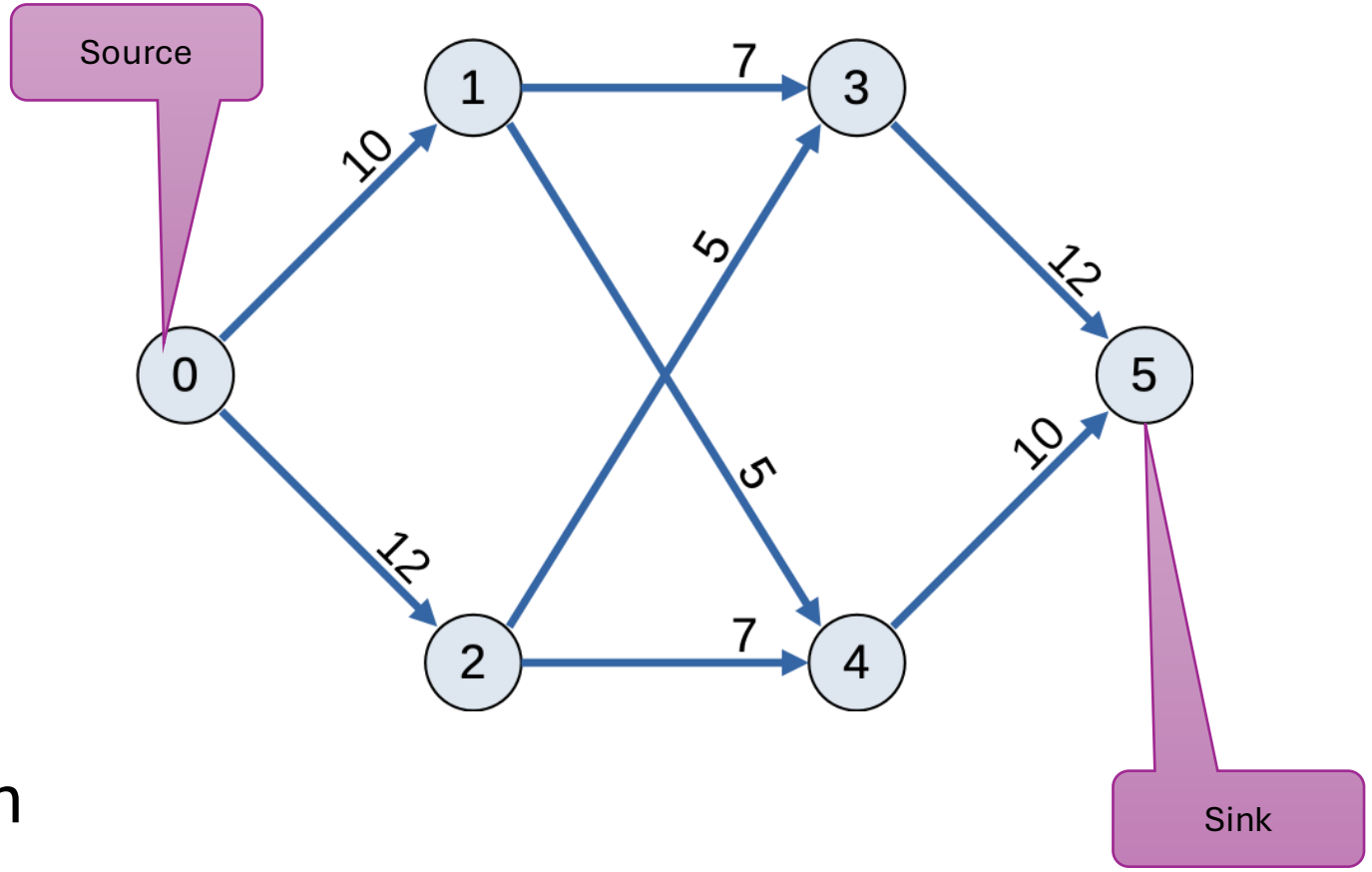
Competitive Programming

Maximum Flow

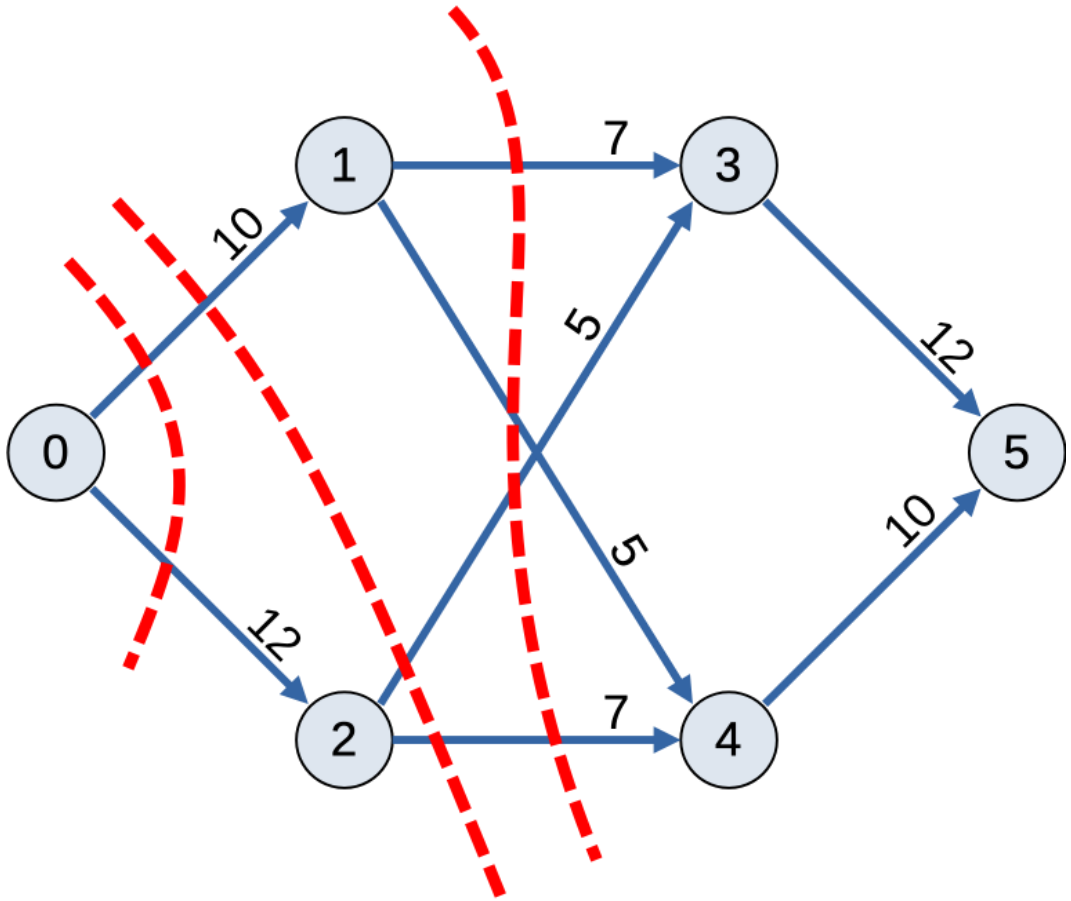
Maximum Flow

- Given a weighted, usually directed graph
- With a source and a sink vertex
- The weights are capacity limits
- Find the maximum amount you can push in parallel from the source to the sink



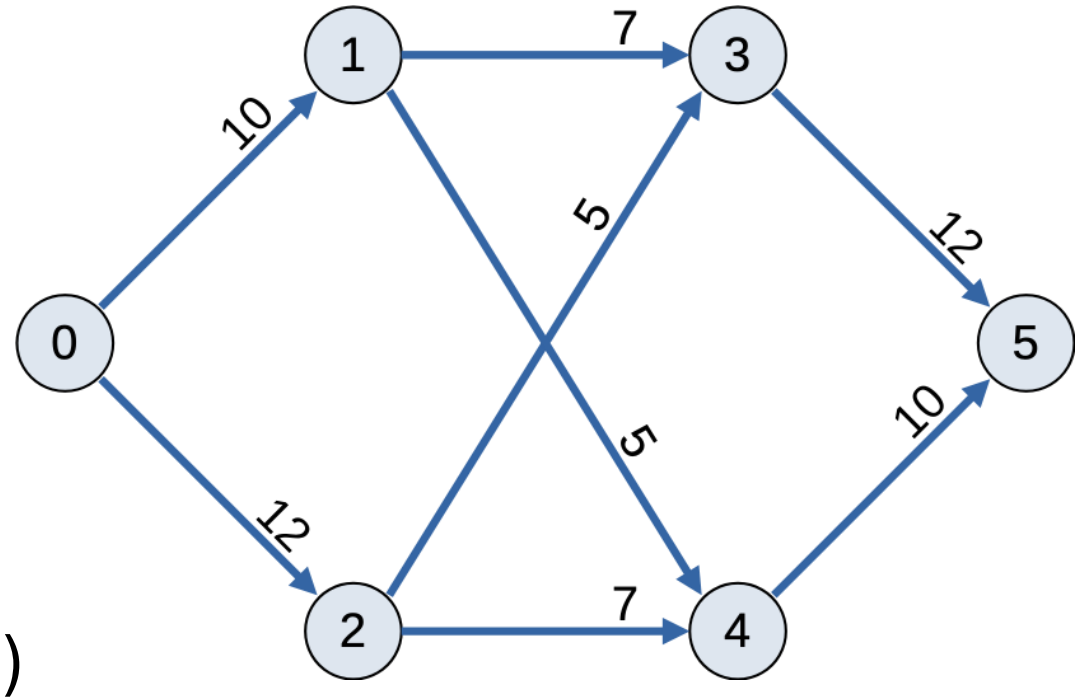
Minimum Cut

- A dual problem
- Lowest total edge weight of a subset of the edges that, if removed, separates the source from the sink



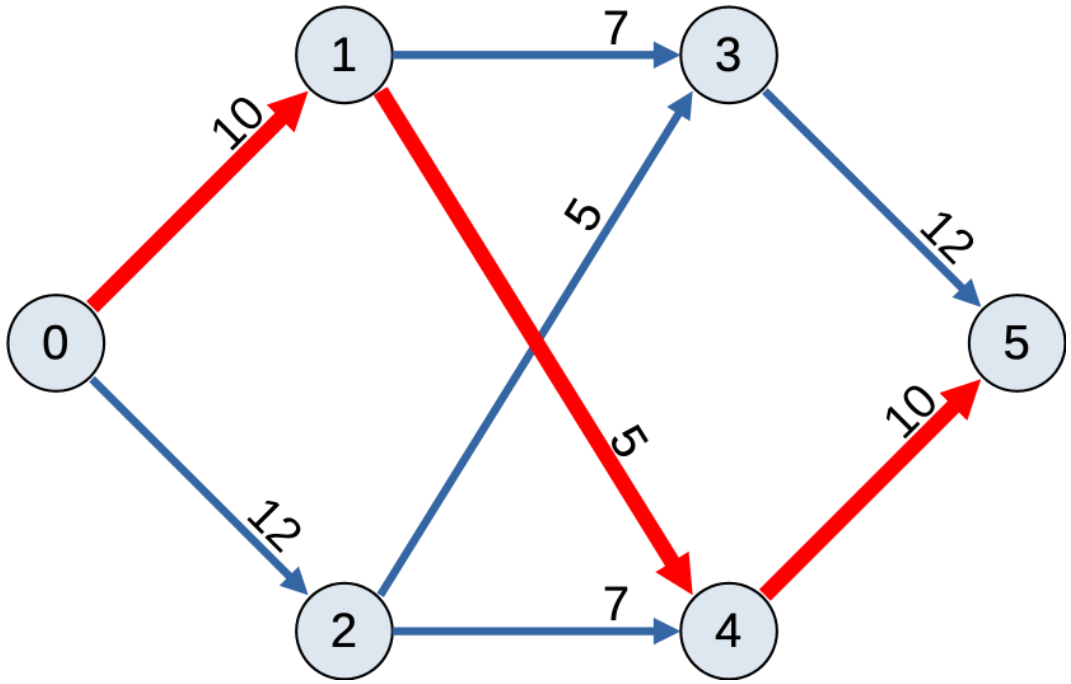
Maximum Flow

- Advanced topic
- Lots of optimization problems can be expressed as max flow or min cut problems
- Suitable for medium-sized graphs (e.g., 100 – 5000 vertices)



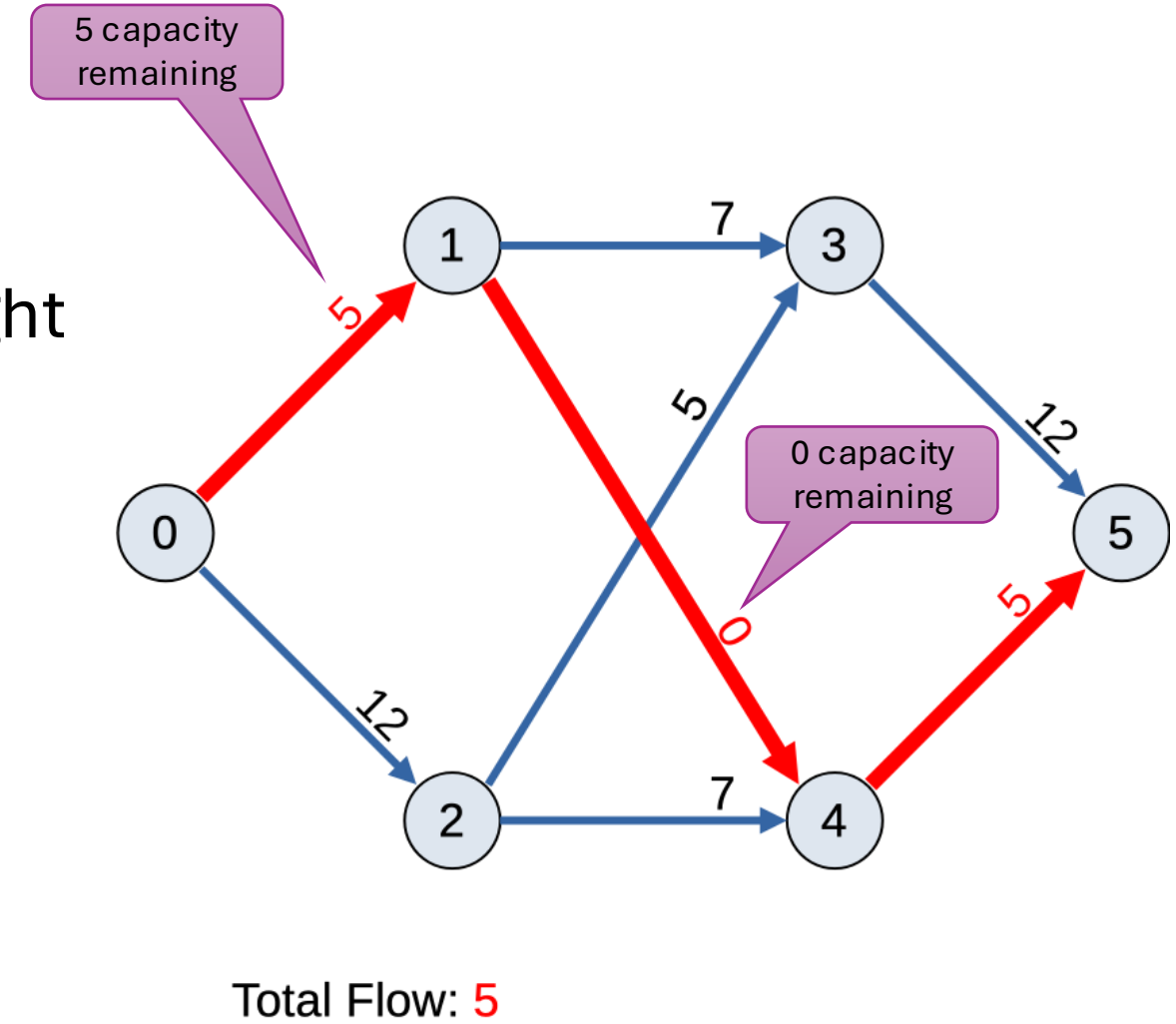
Solving Max Flow

- Find an ***augmenting path***
  - A directed source-to-sink path with positive edge weights



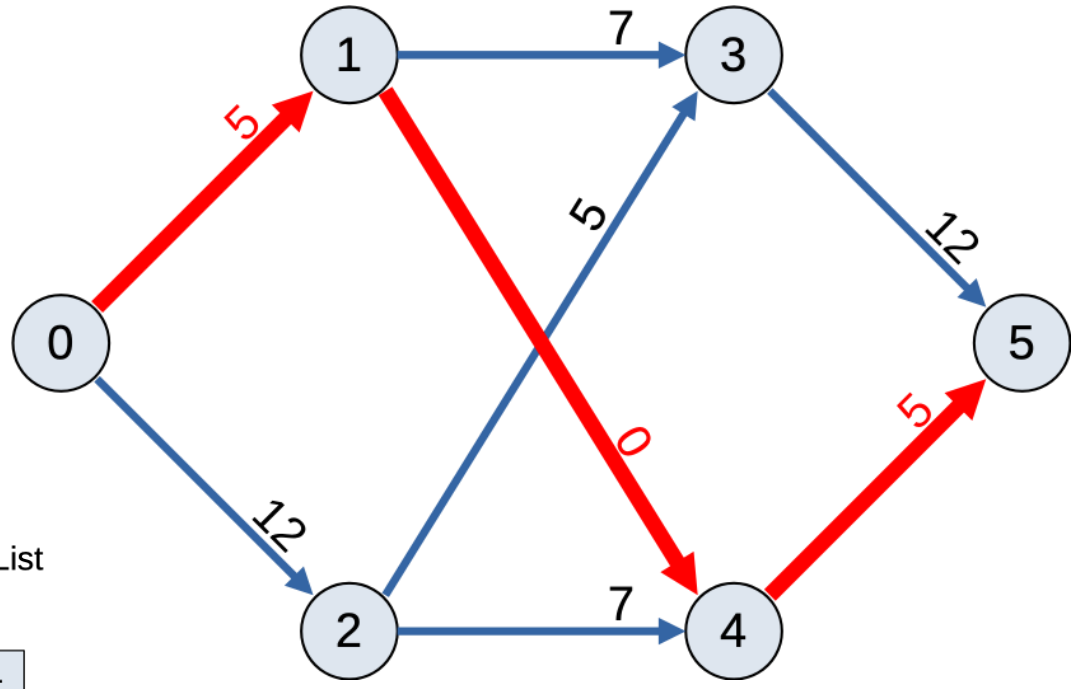
Solving Max Flow

- Increase the flow by the weight of the lowest-weight edge.
  - Saturate that edge
- This reduces the ***residual capacity*** of all edges on the path.



Graph Representation

- Several fields for each edge.
- We can keep one big ***edge list***.
- Adjacency list can reference indices in the edge list.



Edge List

from	to	flow	capacity
0	1	5	10
0	2	0	12
1	3	0	7
1	4	5	5
2	3	0	5
2	4	0	7
3	5	0	12
4	5	5	10

Adjacency List

0	0	1
1	2	3
2	4	5
3	6	
4	7	
5		

Capacity – flow. That's residual capacity.

Finding an Augmenting Path

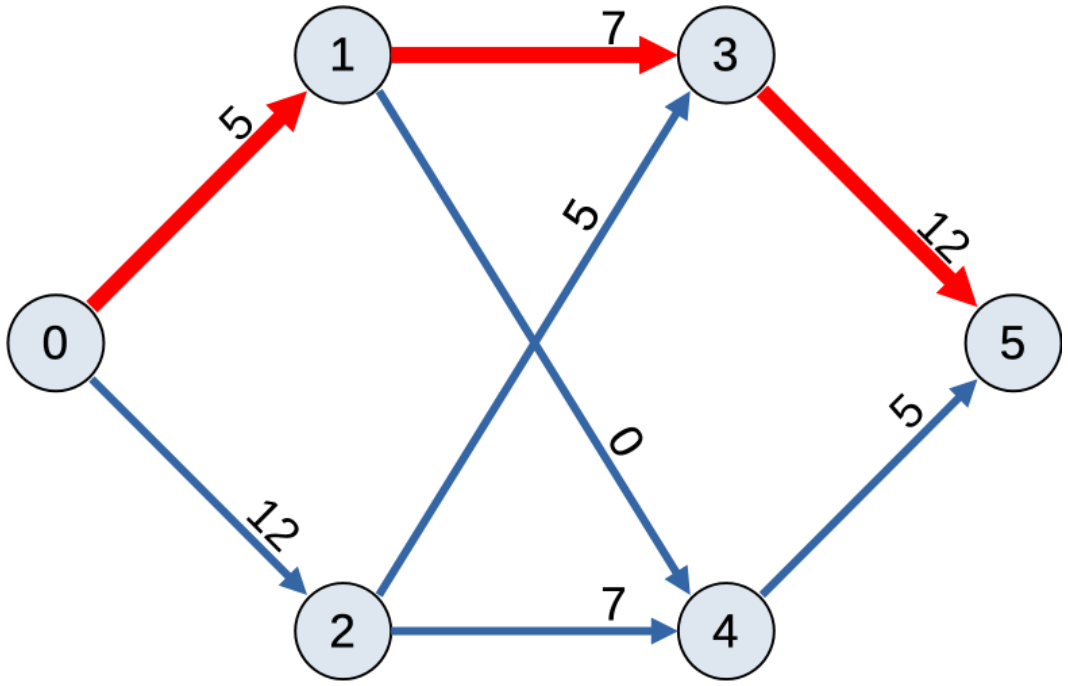
```
int DFS( int i, int f ) {
  if ( visited[ i ] )
    return 0;
  visited[ i ] = true;
  if ( i == sink )
    return f;
  for ( j : adj[ i ] ) {
    if ( elist[ j ].cap - elist[ j ].flow > 0 ) {
      int delta = DFS( elist[ j ].to,
                       min( f, elist[ j ].cap - elist[ j ].flow );
      if ( delta > 0 ) {
         elist[ j ].flow += delta;
         return delta;
      }
    }
  }

  return 0;
}
```

Solving Max Flow

```
int fordFulkerson( ) {
  int f, total = 0;
  visited.assign( V, false );
  while ( ( f = DFS( src, 1e9 ) ) > 0 ) {
    total += f;
    visited.assign( V, false );
  }
}
```

Solving Max Flow

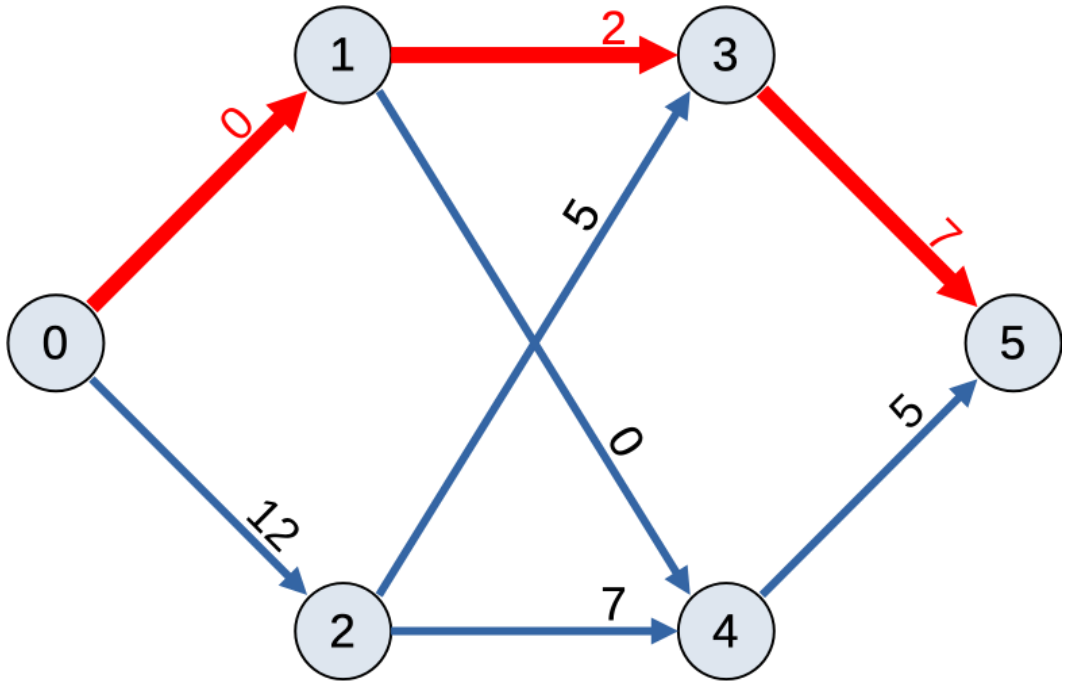
- Find another augmenting path



Total Flow: 5

Solving Max Flow

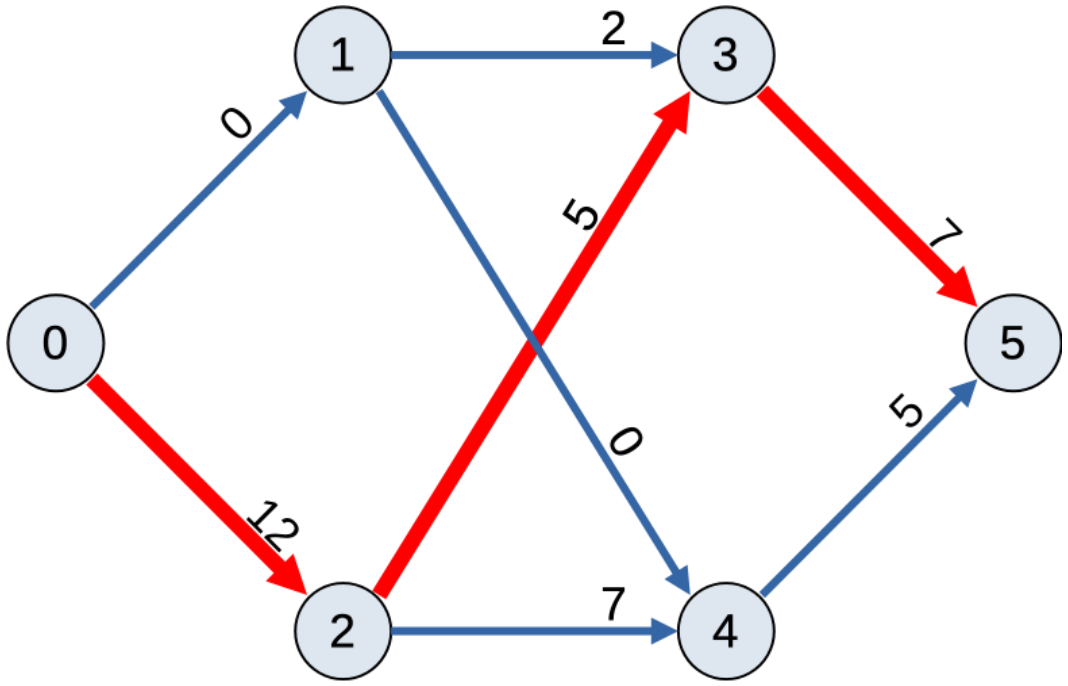
- Saturate its edge with the lowest residual capacity



Total Flow: 5 + 5

Solving Max Flow

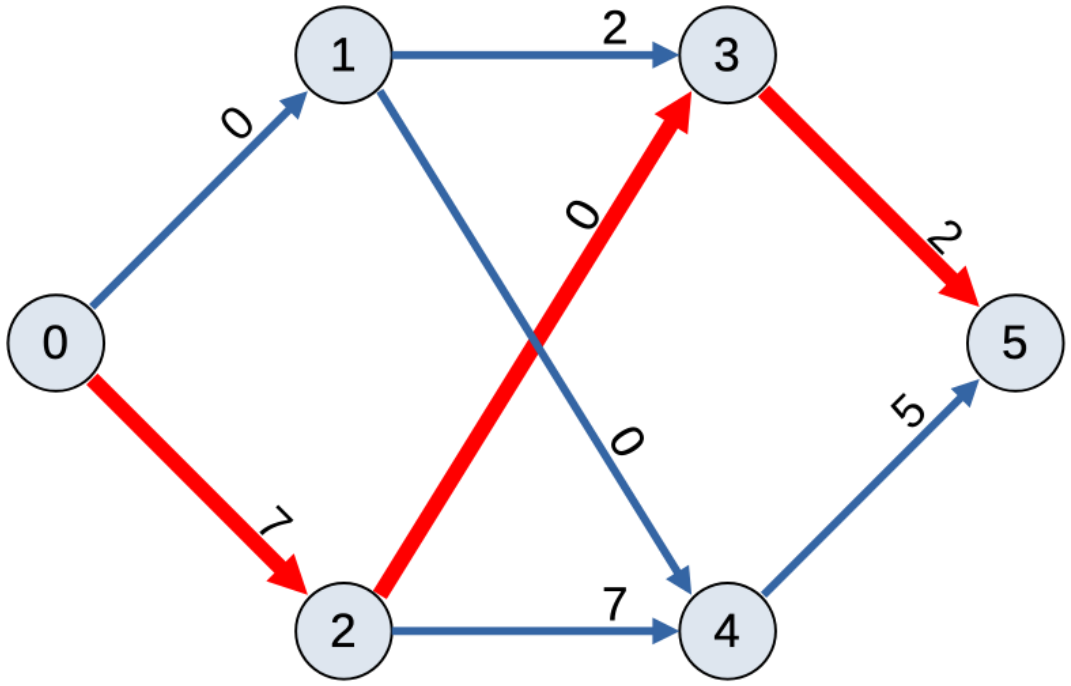
- Find another augmenting path



Total Flow: 5 + 5

Solving Max Flow

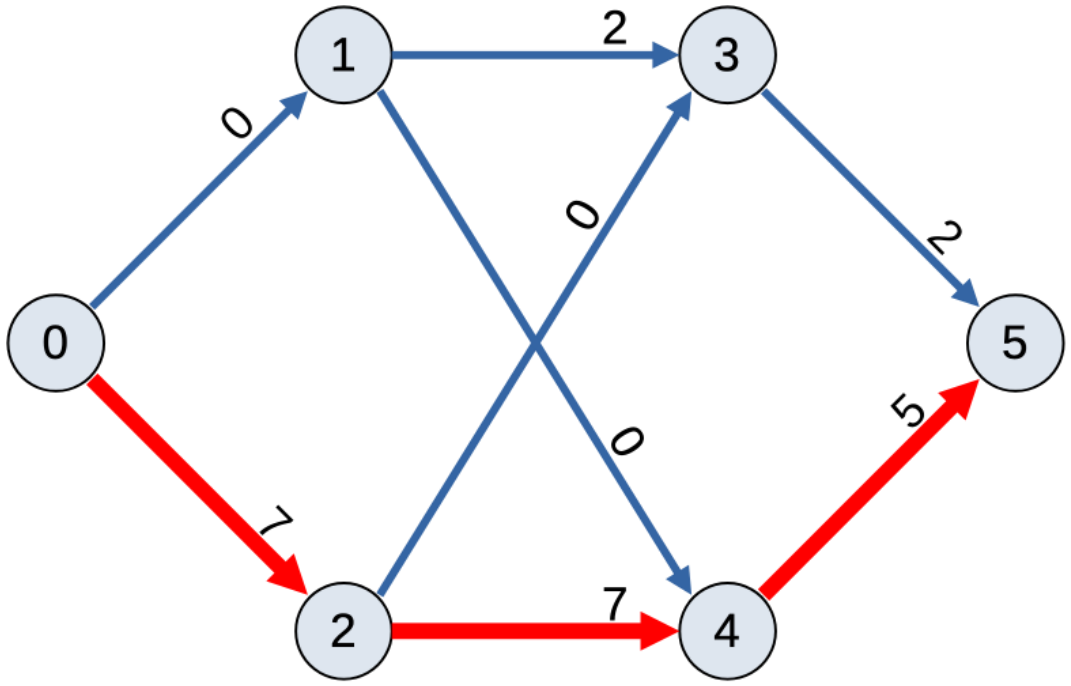
- Saturate its edge with the lowest residual capacity



Total Flow: 5 + 5 + 5

Solving Max Flow

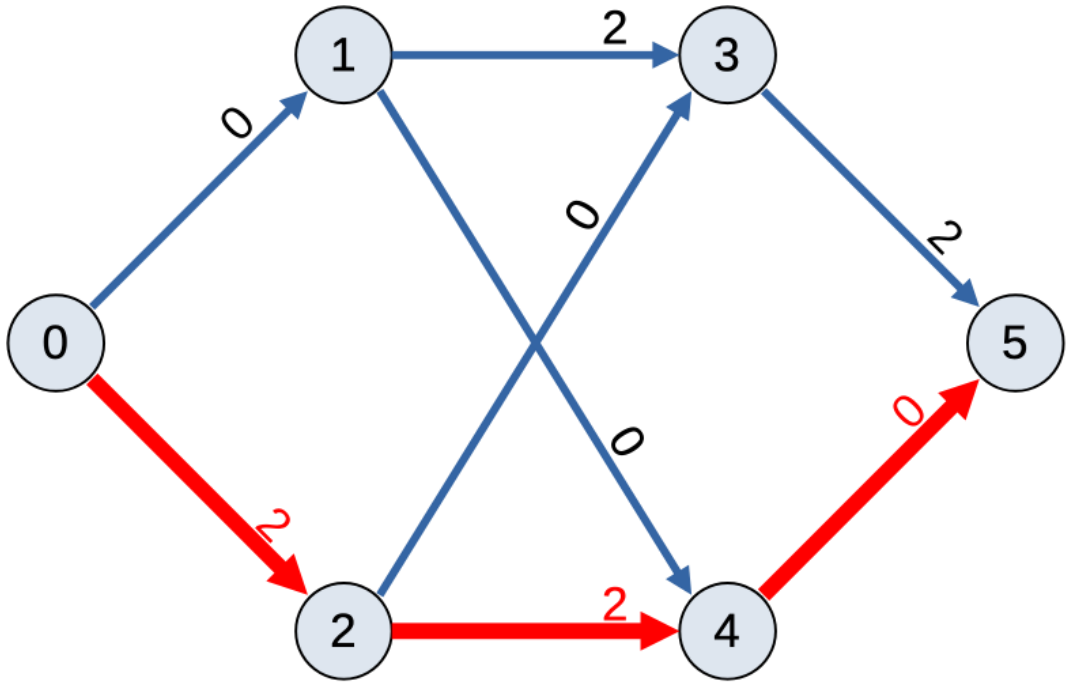
- Find another augmenting path



Total Flow: 5 + 5 + 5

Solving Max Flow

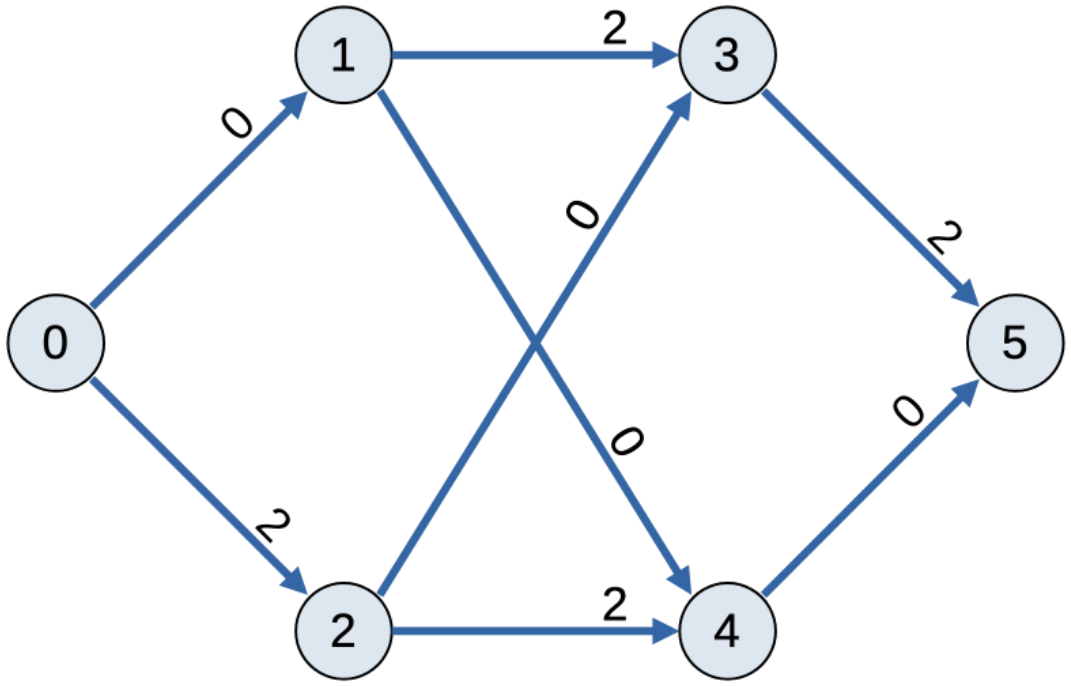
- Saturate its edge with the lowest residual capacity



Total Flow: 5 + 5 + 5 + 5

Solving Max Flow

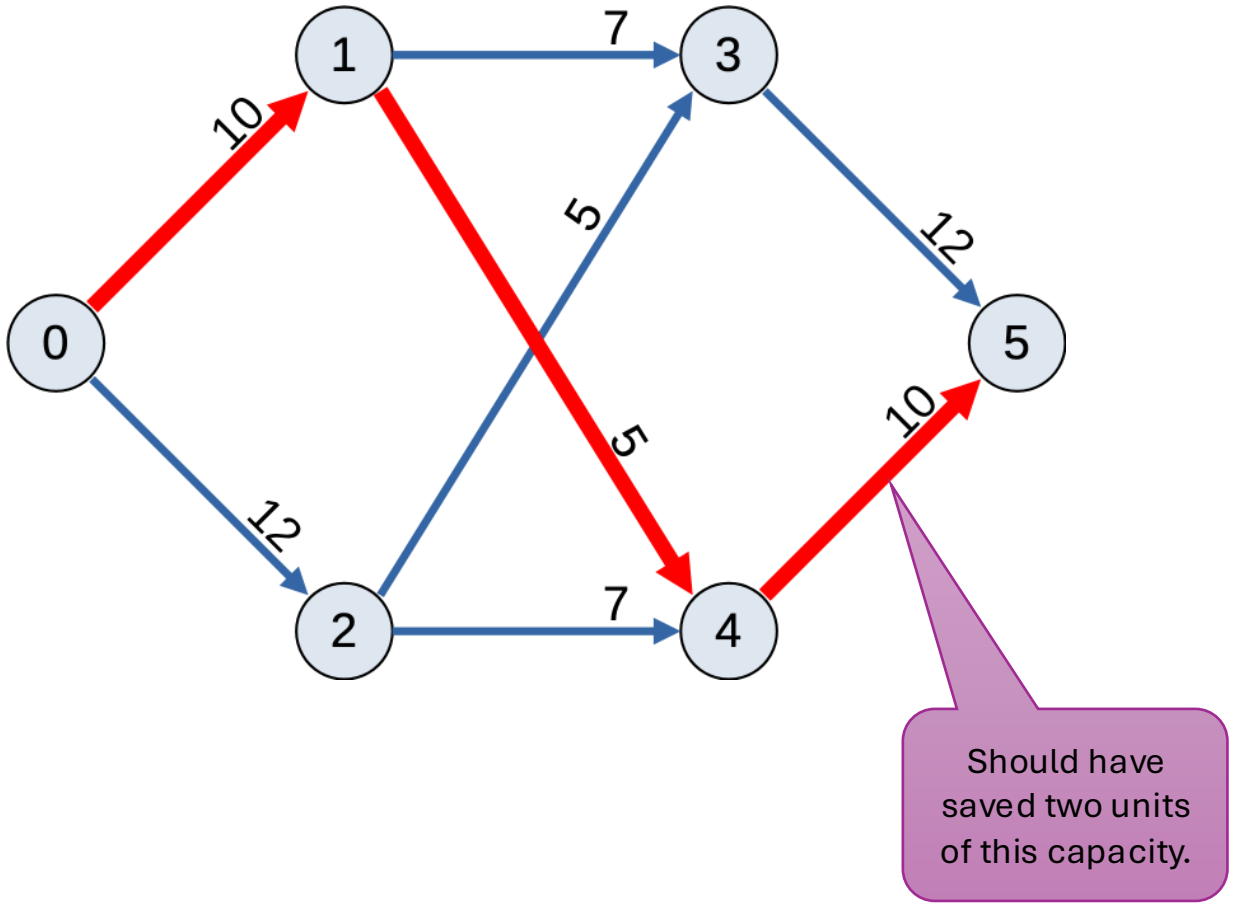
- Find another augmenting path
- Oops. No more augmenting paths.
- But, we're not at max flow yet.



Total Flow: 5 + 5 + 5 + 5

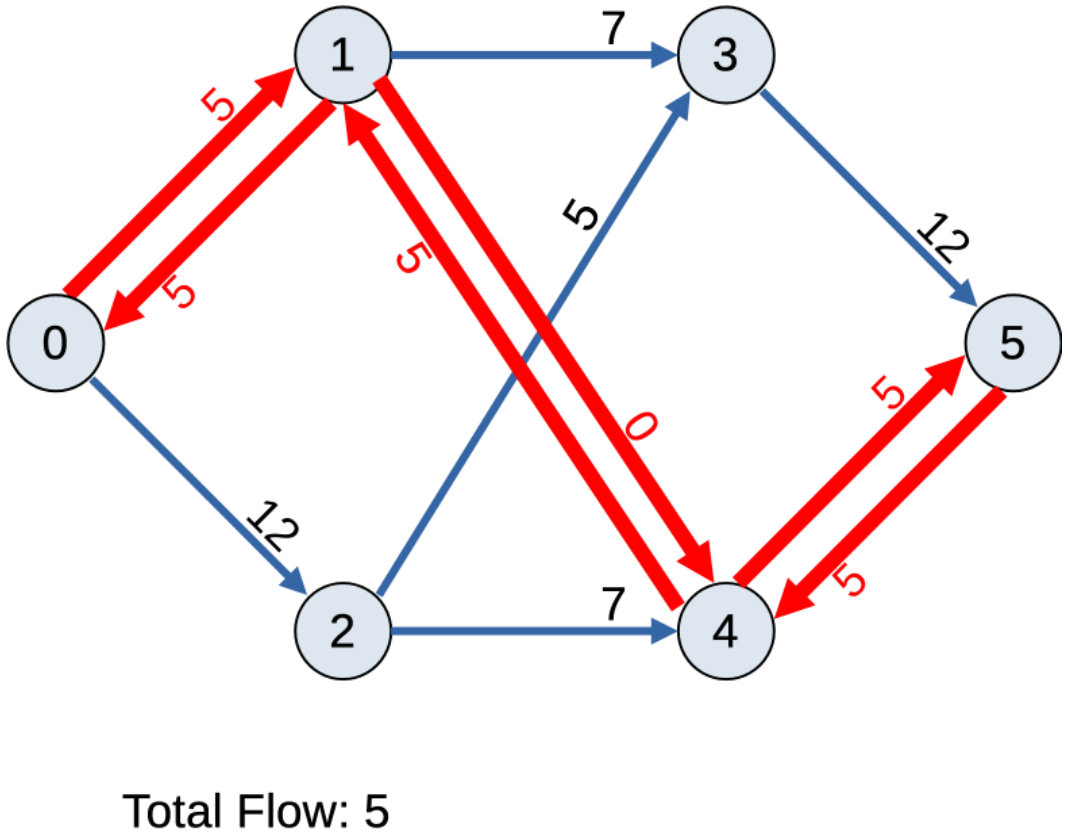
Solving Max Flow

- We made a bad choice back at our first augmenting path.
- But, we don't want to turn this into a search problem.



Solving Max Flow

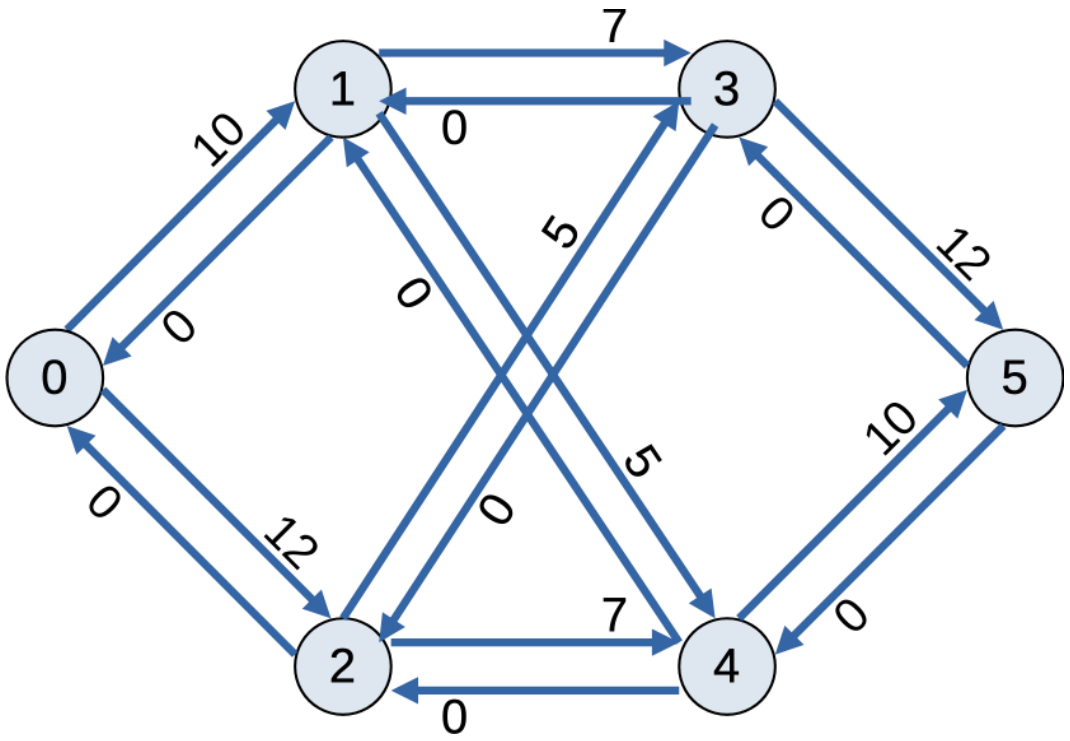
- There are edges we hadn't thought about.
- When we increase the flow in one direction, we create available capacity in the opposite direction.
  - We can choose to un-do the push across that edge.



Total Flow: 5

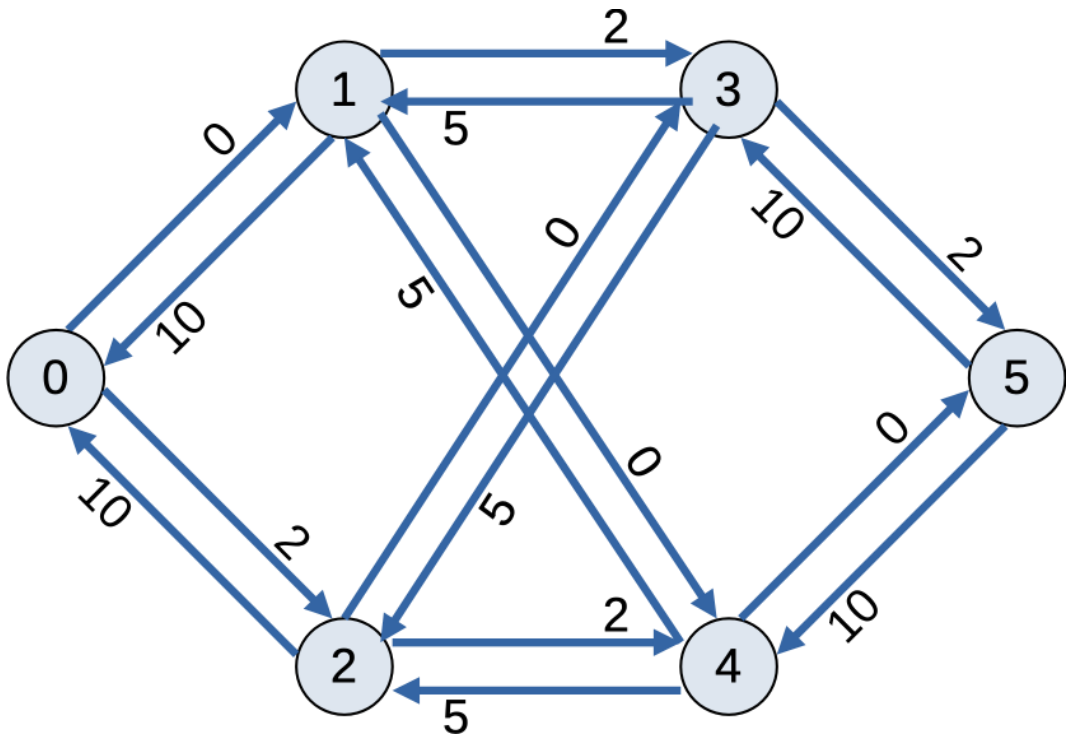
Solving Max Flow

- Really, we should have been doing this with all edges.
- This is what our graph should have looked like at the start.



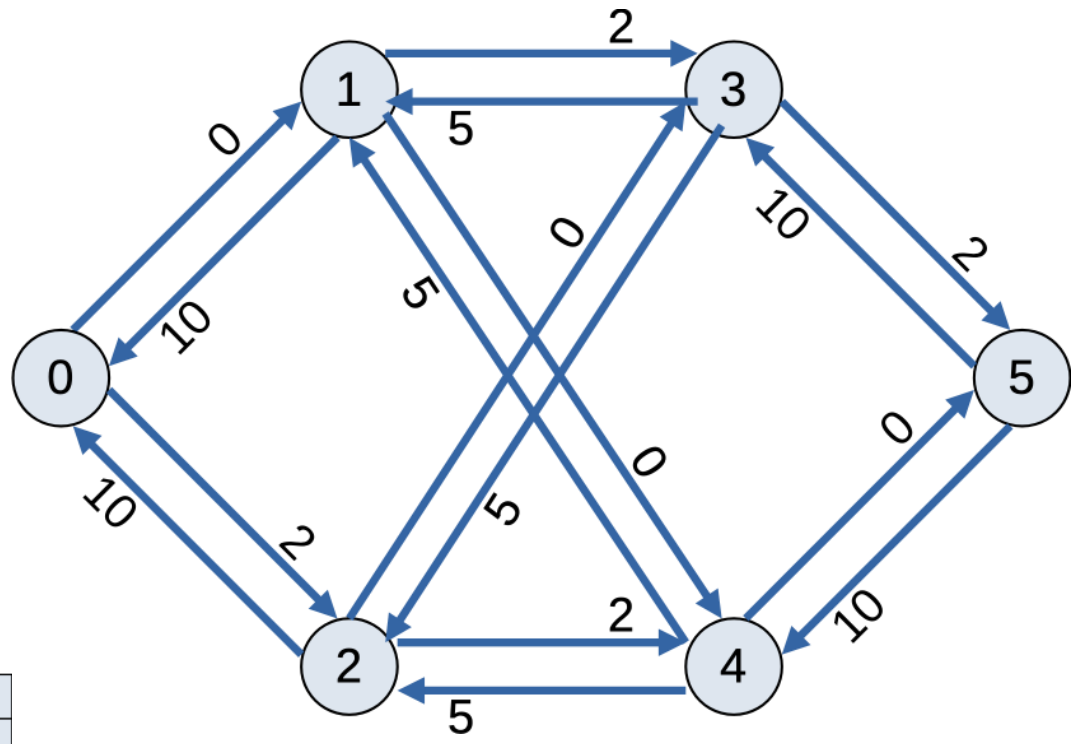
Solving Max Flow

- This is what it would look like now.
  - After saturating the four augmenting paths.



Storing the Graph

- Really, we should have been doing this with all edges.
- Store pairs of edges in even/odd indices.



Edge List

from	to	flow	capacity
0	1	0	10
1	0	-10	0
0	2	0	12
2	0	0	0
1	3	0	7
3	1	0	0
1	4	5	5
4	1	-5	0
…	…	…	…

Adjacency List

0	0	2	
1	1	4	6
2	3	8	10
3	5	9	12
4	7	11	14
5	13	15	

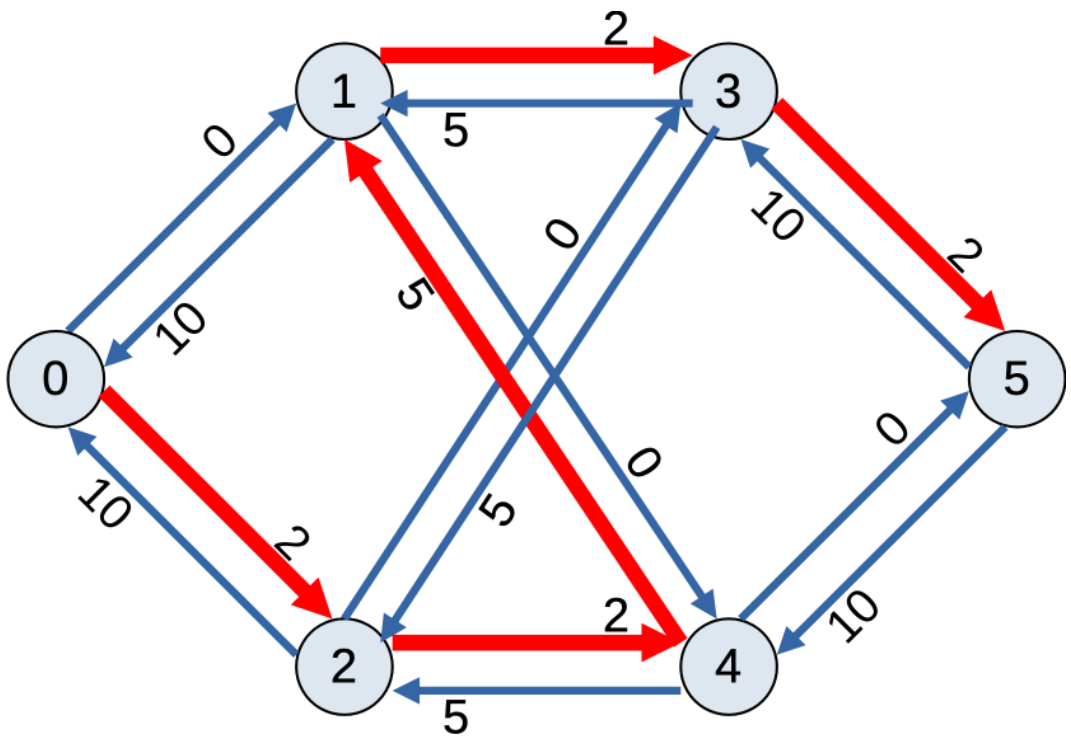
Looks strange, but capacity – flow is still residual capacity.

Finding an Augmenting Path

```
int DFS( int i, int f ) {
  if ( visited[ i ] )
    return 0;
  visited[ i ] = true;
  if ( i == sink )
    return f;
  for ( j : adj[ i ] ) {
    if ( elist[ j ].cap - elist[ j ].flow > 0 ) {
      int delta = DFS( elist[ j ].to,
                       min( f, elist[ j ].cap - elist[ j ].flow );
      if ( delta > 0 ) {
         elist[ j ].flow += delta;
         elist[ j ^ 1 ].flow -= delta;
         return delta;
      }
    }
  }

  return 0;
}
```

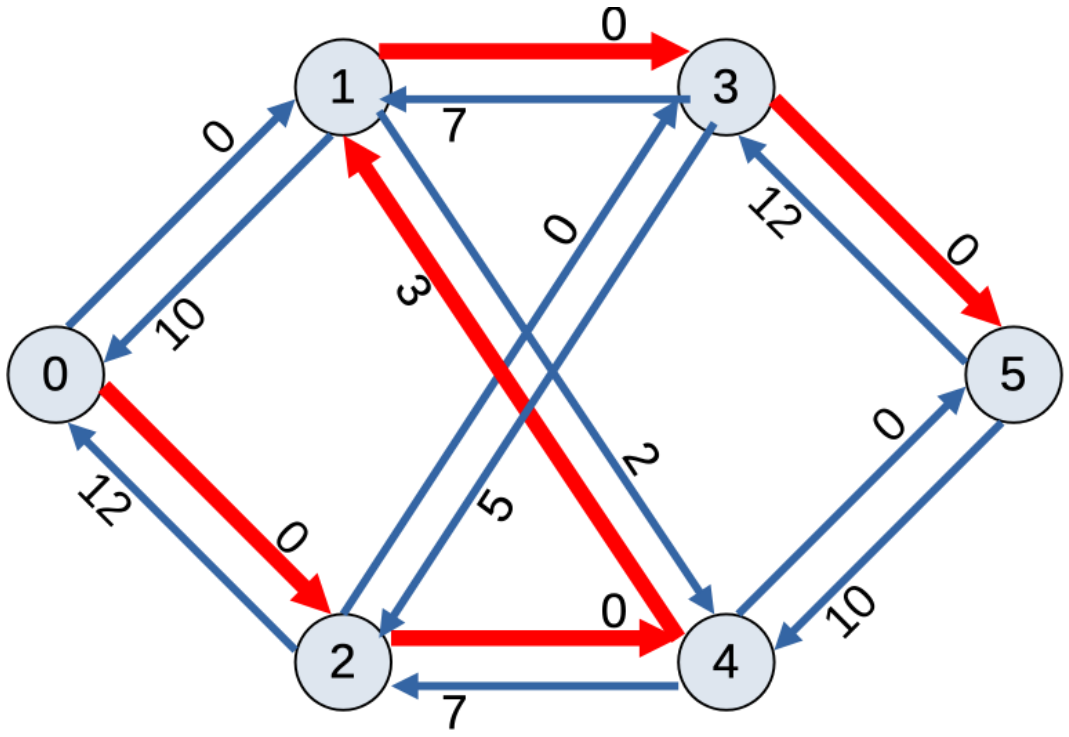
Solving Max Flow

- This lets us find one more augmenting path.
  - Effectively, taking back some of the flow across the 1 → 4 edge.



Solving Max Flow

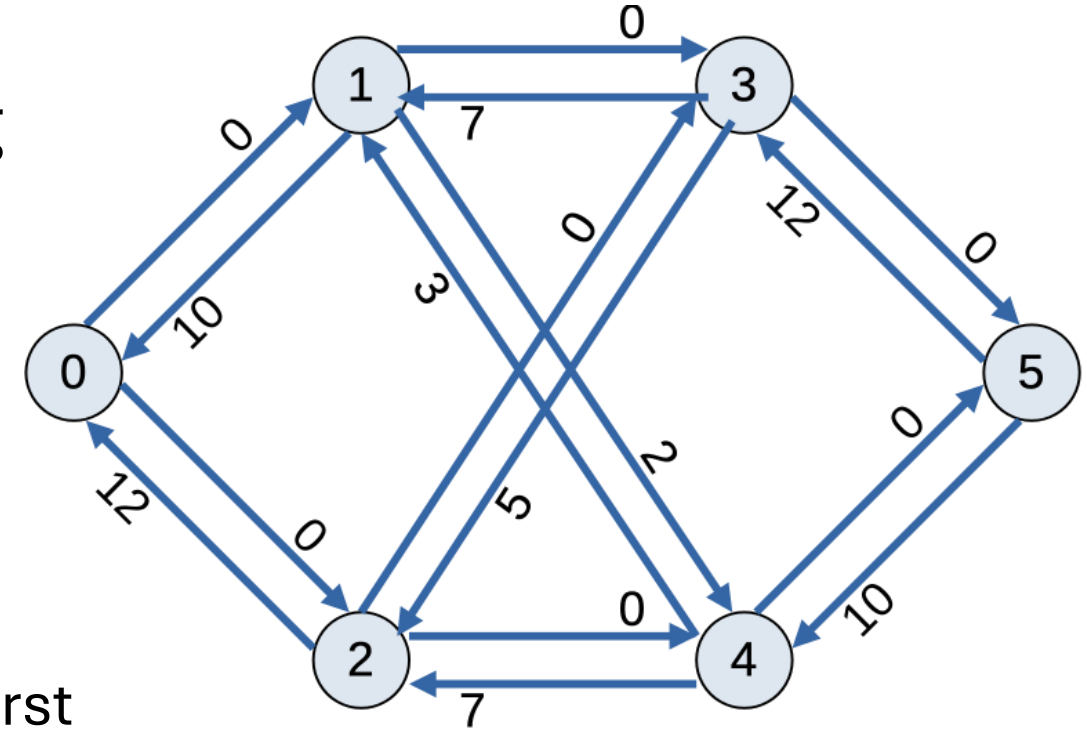
- Saturating this path gets us up to a real max flow.



Total Flow: 5 + 5 + 5 + 5 + 2

Solving Max Flow Faster

- Finding augmenting paths using DFS can be a bad idea.
  - There are some pathological graphs that cause very slow runtime.
- Better to find them starting with BFS
  - Finds shorter augmenting paths first
  - Running time $O(V E^2)$



Total Flow: 5 + 5 + 5 + 5 + 2